

## Инструкция запуска

1. `git clone https://github.com/Gabka365/devops-tms.git`
2. В папке `devops-tms\homework22-23-24` создать `.env` вида:

```
# Common
DB_NAME=CarsStorage

#Api
CONNECTION_STRING=Server=mysqlDb;Port=3306;User Id=root;Password=root;
```

3. Проверить, что порт 8080 свободен
4. В папке `devops-tms\homework22-23-24` выполнить команду:  
`docker compose up -d --build`
5. Get запрос:  
<http://localhost:8080/get/{id}>  
Post запрос:  
<http://localhost:8080/create/mark/{mark}/model/{model}/yearOfRelease/{yearOfRelease}>

## Отчёт

### Теория

1. Чем образ отличается от контейнера?

Образ – это набор файлов с инструкцией запуска программы из них. Контейнер – это сама программа. Подходит аналогия с детским конструктором: образ – это коробка, набор деталей и инструкция. Контейнер – окончательный вариант после сборки всех компонент в единое целое и запуска (например, робота).

2. Зачем в *Compose* именованный том для БД и чем он принципиально отличается от *bind-mount* для каталога с кодом при разработке?

Именованный том для БД нужен, чтобы обеспечить постоянное, изолированное и высокопроизводительное хранение данных, которые должны переживать перезапуски и удаление контейнера, причём Docker полностью управляет его расположением и безопасностью. Bind mount в Docker – это прямая привязка папки или файла с вашего компьютера (хоста) внутрь контейнера для его конфигурации.

3. Зачем две *bridge*-сети между прокси, приложением и БД?

Две подсети позволяют повысить безопасность сети. Если подсеть с прокси оказалась взломана, то доступ к БД получен не будет.

## Образ приложения и сборка

### 1. docker compose build:

```
PS C:\Users\maxpo\devops-tms\homework22-23-24> docker compose build
[+] Building 10.1s (20/20) FINISHED                                docker:desktop-linux
=> [cars_api internal] load build definition from Dockerfile
=> transferring dockerfile: 799B
=> [cars_api internal] load metadata for mcr.microsoft.com/dotnet/sdk:10.0
=> [cars_api internal] load metadata for mcr.microsoft.com/dotnet/aspnet:10.0
=> [cars_api internal] load .dockerignore
=> transferring context: 2B
=> [cars_api build 1/7] FROM mcr.microsoft.com/dotnet/sdk:10.0@sha256:c0790639332692a0d56cdd81ed581cfd24d040d9839764c13899
=> resolve mcr.microsoft.com/dotnet/sdk:10.0@sha256:c0790639332692a0d56cdd81ed581cfd24d040d9839764c138994866df89a3b6
=> [cars_api internal] load build context
=> transferring context: 2.55kB
=> [cars_api base 1/3] FROM mcr.microsoft.com/dotnet/aspnet:10.0@sha256:8c0b6857eab7b2aa57884c839bf4678414606bd7d17370f18a
=> resolve mcr.microsoft.com/dotnet/aspnet:10.0@sha256:8c0b6857eab7b2aa57884c839bf4678414606bd7d17370f18a842ac5cf414711
=> CACHED [cars_api build 2/7] WORKDIR /src
=> CACHED [cars_api build 3/7] COPY [Api/Api.Api.csproj, Api/]
=> CACHED [cars_api build 4/7] RUN dotnet restore ".\Api\Api.csproj"
=> [cars_api build 5/7] COPY . .
=> [cars_api build 6/7] WORKDIR /src/Api
=> [cars_api build 7/7] RUN dotnet build ".\Api.csproj" -c Release -o /app/build /p:UseAppHost=false /p:GenerateAssemblyIn
=> [cars_api publish 1/1] RUN dotnet publish ".\Api.csproj" -c Release -o /app/publish /p:UseAppHost=false /p:GenerateAsse
=> CACHED [cars_api base 2/3] WORKDIR /app
=> CACHED [cars_api base 3/3] RUN apt-get update && apt-get install -y curl
=> CACHED [cars_api final 1/2] WORKDIR /app
=> CACHED [cars_api final 2/2] COPY --from=publish /app/publish .
=> [cars_api] exporting to image
=> exporting layers
=> exporting manifest sha256:aa297d08ac1b619b790cd4914ecba84ffef89d02659b2390ef3c8512f8b19424
=> exporting config sha256:eb7954c0f6d5b384b53c816aa5ce834c78b789620f79f970f3933b1d5ff5768
=> exporting attestation manifest sha256:cfe263cc8d5260758abe45eb13ac61d5ba39f639a698c5e2f47111acfc3d1bb
=> exporting manifest list sha256:c7f432ad06f6ca8bbde926c8e93505d28ff0ec489b004087c521e1bb96012c62
=> naming to docker.io/library/homework22-23-24-cars_api:latest
=> unpacking to docker.io/library/homework22-23-24-cars_api:latest
=> [cars_api] resolving provenance for metadata file
PS C:\Users\maxpo\devops-tms\homework22-23-24>
```

### 2. docker compose build --no-cache:

```
PS C:\Users\maxpo\devops-tms\homework22-23-24> docker compose build --no-cache
[+] Building 28.6s (20/20) FINISHED                                docker:desktop-linux
=> [cars_api internal] load build definition from Dockerfile
=> transferring dockerfile: 799B
=> [cars_api internal] load metadata for mcr.microsoft.com/dotnet/sdk:10.0
=> [cars_api internal] load metadata for mcr.microsoft.com/dotnet/aspnet:10.0
=> [cars_api internal] load .dockerignore
=> transferring context: 2B
=> [cars_api build 1/7] FROM mcr.microsoft.com/dotnet/sdk:10.0@sha256:c0790639332692a0d56cdd81ed581cfd24d040d9839764c138994866df89a3b6
=> resolve mcr.microsoft.com/dotnet/sdk:10.0@sha256:c0790639332692a0d56cdd81ed581cfd24d040d9839764c138994866df89a3b6
=> [cars_api base 1/3] FROM mcr.microsoft.com/dotnet/aspnet:10.0@sha256:8c0b6857eab7b2aa57884c839bf4678414606bd7d17370f18a842ac5cf414711
=> resolve mcr.microsoft.com/dotnet/aspnet:10.0@sha256:8c0b6857eab7b2aa57884c839bf4678414606bd7d17370f18a842ac5cf414711
=> [cars_api internal] load build context
=> transferring context: 2.55kB
=> CACHED [cars_api build 2/7] WORKDIR /src
=> CACHED [cars_api base 2/3] WORKDIR /app
=> [cars_api base 3/3] RUN apt-get update && apt-get install -y curl
=> [cars_api build 3/7] COPY [Api/Api.Api.csproj, Api/]
=> [cars_api build 4/7] RUN dotnet restore ".\Api\Api.csproj"
=> [cars_api build 5/7] COPY . .
=> [cars_api build 6/7] WORKDIR /src/Api
=> [cars_api build 7/7] RUN dotnet build ".\Api.csproj" -c Release -o /app/build /p:UseAppHost=false /p:GenerateAssemblyInfo=false
=> [cars_api final 1/2] WORKDIR /app
=> [cars_api publish 1/1] RUN dotnet publish ".\Api.csproj" -c Release -o /app/publish /p:UseAppHost=false /p:GenerateAssemblyInfo=false
=> [cars_api final 2/2] COPY --from=publish /app/publish .
=> [cars_api] exporting to image
=> exporting layers
=> exporting manifest sha256:0a078f1cb4f08dec8eeffa69945ae36d7918e078d099fa6093e7cf12011e7a4
=> exporting config sha256:5863daad67112dad55994622cb13879242f9ac01a6aa97ae2567807786dc55db
=> exporting attestation manifest sha256:85f6b0c72ac832fa2b0b9309f26d1b8b956e1e3bd76bf5b462cac1e85b5df100
=> exporting manifest list sha256:22bc3ac459304b1c1e4ec636fddc6445294df27f13258fdd57f9f43418cb23b
=> naming to docker.io/library/homework22-23-24-cars_api:latest
=> unpacking to docker.io/library/homework22-23-24-cars_api:latest
=> [cars_api] resolving provenance for metadata file
PS C:\Users\maxpo\devops-tms\homework22-23-24>
```

### 3. Размер docker-образа для API:

| REPOSITORY                | TAG    | IMAGE ID     | CREATED       | SIZE  |
|---------------------------|--------|--------------|---------------|-------|
| homework22-23-24-cars_api | latest | 22bc3ac45930 | 3 minutes ago | 472MB |

Multi-stage даёт финальный образ намного меньше, чем одностадийный на SDK, потому что в него копируются только собранные бинарные файлы, а весь компилятор, заголовки, инструменты сборки и SDK остаются на промежуточной стадии.

## Docker-compose как единый стек

### 1. Описание docker-compose:

Реализована трёхсервисная архитектура с полной изоляцией сетевых сегментов. MySQL (mysqlDb) находится только в сети backend, Nginx – только в frontend, а само приложение (cars\_api) подключено к обеим сетям,

выступая единственным мостом между прокси и базой данных. Даже при компрометации Nginx злоумышленник не получит доступа к MySQL, так как между ними нет прямого маршрута. Для сохранения состояния используются два типа томов: именованный том `datafiles` для данных MySQL (чтобы БД переживала перезапуски и удаление контейнера) и `bind mount` `./logs-dotnet:/app/logs-dotnet` для логов приложения (удобно для отладки на хосте). Логирование всех сервисов унифицировано через якорь `x-logging` (ротация JSON-логов до 10 МБ, 5 файлов со сжатием). Для надёжности настроены `healthcheck` каждого сервиса: MySQL проверяется через `mysqladmin ping`, API – через `curl` к `/health`, а Nginx стартует только после готовности API благодаря `depends_on` с проверкой условия `service_healthy`. Порт 8080 Nginx пробрасывается на хост, обеспечивая единую точку входа.

## 2. Поднятие стека:

```
PS C:\Users\maxpo\devops-tms\homework22-23-24> docker compose up -d
[+] Running 3/3
  ✓ Container mysqlDb    Healthy
  ✓ Container cars-api    Healthy
  ✓ Container nginx       Running
PS C:\Users\maxpo\devops-tms\homework22-23-24> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS              PORTS
cars-api            homework22-23-24-cars_api  "dotnet Api.dll"      cars_api   45 seconds ago  Up 42 seconds (healthy)  8080/tcp
mysqlDb            mysql:latest         "docker-entrypoint.s..." mysqlDb    20 hours ago   Up 23 minutes (healthy)  3306/tcp, 33060/tcp
nginx              nginx:alpine         "/docker-entrypoint..." nginx      20 hours ago   Up 36 seconds          80/tcp, 0.0.0.0:8080->8080/tcp
```

## 3. Перемонтирование сервиса:

```
PS C:\Users\maxpo\devops-tms\homework22-23-24> docker compose up -d --build nginx
[+] Building 11.2s (26/26) FINISHED
=> [cars_api internal] load build definition from Dockerfile
=> [cars_api internal] load metadata for mcr.microsoft.com/dotnet/aspnet:10.0
=> [cars_api internal] load metadata for mcr.microsoft.com/dotnet/sdk:10.0
=> [cars_api internal] load .dockerignore
=> [cars_api internal] load build context
=> [cars_api build 1/7] FROM mcr.microsoft.com/dotnet/sdk:10.0@sha256:c0790639332692a0d56cdd81ed581cf24d040d9839764c138994866df89a3b6
=> [cars_api build 2/7] FROM mcr.microsoft.com/dotnet/aspnet:10.0@sha256:8c0b6857eab7b2aa57884c839bf4678414606bd7d17370f18a842ac5cf414711
=> [cars_api build 3/7] COPY [Api/Api.Api.csproj, Api/]
=> [cars_api build 4/7] RUN dotnet restore ".\Api\Api.csproj"
=> [cars_api build 5/7] COPY .
=> [cars_api build 6/7] WORKDIR /src/Api
=> [cars_api build 7/7] RUN dotnet build ".\Api.csproj" -c Release -o /app/build /p:UseAppHost=false /p:GenerateAssemblyInfo=false
=> [cars_api publish 1/1] RUN dotnet publish ".\Api.csproj" -c Release -o /app/publish /p:UseAppHost=false /p:GenerateAssemblyInfo=false
=> [cars_api base 2/3] WORKDIR /app
=> [cars_api base 3/3] RUN apt-get update && apt-get install -y curl
=> [cars_api final 1/2] WORKDIR /app
=> [cars_api final 2/2] COPY --from=publish /app/publish .
=> [cars_api] exporting to image
=> [cars_api] exporting layers
=> [cars_api] exporting manifest sha256:8a078f1fcb4f08dec0eef6a69945ae36d7918e078d099fa6093e7cf12011e7a4
=> [cars_api] exporting config sha256:5863daa6d7112dad55994622cb13879242f9ac01a6aa97ae2567807706dc55db
=> [cars_api] exporting attestation manifest sha256:a28f9a9ec67264dc2103911bba7ab5c9e8b695672ed6d2c59679774d5fc20f688
=> [cars_api] exporting manifest list sha256:df385bebfb839b655f905175e2bb9438a6ab3e8fcd2418e2f2eb8a876f97776
=> [cars_api] naming to docker.io/library/homework22-23-24-cars_api:latest
=> [cars_api] unpacking to docker.io/library/homework22-23-24-cars_api:latest
=> [cars_api] resolving provenance for metadata file
[+] Running 3/3
  ✓ Container mysqlDb    Healthy
  ✓ Container nginx       Started
  ✓ Container cars-api    Healthy
```

## Наблюдаемость и эксплуатация

### 1. Логи API:

```
PS C:\Users\maxpo\devops-tms\homework22-23-24> docker logs cars-api
20:10:46 [INF] HTTP GET /health responded 200 in 13.3275 ms
20:11:16 [INF] HTTP GET /health responded 200 in 0.6524 ms
20:11:46 [INF] HTTP GET /health responded 200 in 0.1361 ms
20:12:16 [INF] HTTP GET /health responded 200 in 0.2499 ms
20:12:46 [INF] HTTP GET /health responded 200 in 0.1263 ms
20:13:17 [INF] HTTP GET /health responded 200 in 0.1327 ms
20:13:47 [INF] HTTP GET /health responded 200 in 0.1879 ms
20:14:17 [INF] HTTP GET /health responded 200 in 0.1175 ms
20:14:47 [INF] HTTP GET /health responded 200 in 0.1862 ms
20:15:17 [INF] HTTP GET /health responded 200 in 0.0956 ms
20:15:47 [INF] HTTP GET /health responded 200 in 0.1056 ms
```

### 2. Инспекция контейнера API:

Контейнер подключен к двум сетям. Оба IP назначены из разных подсетей (172.19.x.x и 172.21.x.x), что подтверждает физическую изоляцию двух bridge-сетей. В каждой сети у контейнера есть DNS-alias: cars-api, cars\_api, c05f3325e3c7 – это позволяет другим сервисам обращаться к нему по имени в пределах своей сети.

```
"Networks": {
  "homework22-23-24_backend": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": [
      "cars-api",
      "cars_api"
    ],
    "MacAddress": "02:42:ac:13:00:02",
    "DriverOpts": null,
    "NetworkID": "14195fb4d0833cd778e54d437d4",
    "EndpointID": "3c05f1e9e5a6f307e61767da7a",
    "Gateway": "172.19.0.1",
    "IPAddress": "172.19.0.2",
    "IPPrefixLen": 16,
    "IPv6Gateway": "",
    "GlobalIPv6Address": "",
    "GlobalIPv6PrefixLen": 0,
    "DNSNames": [
      "cars-api",
      "cars_api",
      "c05f3325e3c7"
    ]
  },
  "homework22-23-24_frontend": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": [
```

В секции Mounts и HostConfig.Binds виден один bind mount. Папка logs-dotnet на хосте привязана к /app/logs-dotnet внутри контейнера. Права – чтение/запись (rw), что позволяет приложению писать логи прямо на хост для отладки:

```
"Mounts": [
  {
    "Type": "bind",
    "Source": "C:\\Users\\maxpo\\devops-tms\\homework22-23-24\\logs-dotnet",
    "Destination": "/app/logs-dotnet",
    "Mode": "rw",
    "RW": true,
    "Propagation": "rprivate"
  }
],
```

В секции LogConfig задана ротация логов. Сжимать после ротации, максимальное количество файлов – 5 штук, делать ротацию при превышении 10Мб.

```
"LogConfig": {
  "Type": "json-file",
  "Config": {
    "compress": "true",
    "max-file": "5",
    "max-size": "10m"
  }
}
```

### 3. Генерация нагрузки:

```
admin@some:~/devops-tms/homework22-23-24$ ./data.sh
=== Итерация 1 ===
Создана запись с ID: 1
Полученные данные: {"id":1,"mark":"Toyota","model":"Camry","yearOfRelease":2020}

=== Итерация 2 ===
Создана запись с ID: 2
Полученные данные: {"id":2,"mark":"Toyota","model":"Camry","yearOfRelease":2020}

=== Итерация 3 ===
Создана запись с ID: 3
Полученные данные: {"id":3,"mark":"Toyota","model":"Camry","yearOfRelease":2020}

=== Итерация 4 ===
Создана запись с ID: 4
Полученные данные: {"id":4,"mark":"Toyota","model":"Camry","yearOfRelease":2020}

=== Итерация 5 ===
Создана запись с ID: 5
Полученные данные: {"id":5,"mark":"Toyota","model":"Camry","yearOfRelease":2020}

=== Итерация 6 ===
Создана запись с ID: 6
Полученные данные: {"id":6,"mark":"Toyota","model":"Camry","yearOfRelease":2020}

=== Итерация 7 ===
Создана запись с ID: 7
Полученные данные: {"id":7,"mark":"Toyota","model":"Camry","yearOfRelease":2020}

=== Итерация 8 ===
Создана запись с ID: 8
Полученные данные: {"id":8,"mark":"Toyota","model":"Camry","yearOfRelease":2020}
```

### Ресурсы под нагрузкой:

| CONTAINER ID | NAME     | CPU % | MEM USAGE / LIMIT   | MEM % | NET I/O       | BLOCK I/O  | PIDS |
|--------------|----------|-------|---------------------|-------|---------------|------------|------|
| 94ed3036cb83 | cars-api | 2.10% | 182.9MiB / 7.754GiB | 2.30% | 402kB / 350kB | 136MB / 0B | 22   |

### Ресурсы без нагрузки:

| CONTAINER ID | NAME     | CPU % | MEM USAGE / LIMIT | MEM % | NET I/O       | BLOCK I/O  | PIDS |
|--------------|----------|-------|-------------------|-------|---------------|------------|------|
| 94ed3036cb83 | cars-api | 0.01% | 182MiB / 7.754GiB | 2.29% | 499kB / 443kB | 136MB / 0B | 21   |

## Тема и персистентность

### 1. Успешно созданная запись в БД:

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/create/mark/bmw/model/e38/yearOfRelease/2023`. The 'Params' tab is active, showing a table with one row: 'Key' and 'Value'. The status bar at the bottom indicates a 200 OK response with 1928 ms latency and 179 B body size. The 'Body' tab is also visible, showing a 'Pretty' view of the response.

| Key | Value |
|-----|-------|
| Key | Value |

200 OK 1928 ms 179 B

### 2. После `docker compose down` / `docker compose up` запрос по id проходит успешно:

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/get/14`. The 'Params' tab is active, showing a table with one row: 'Key' and 'Value'. The status bar at the bottom indicates a 200 OK response with 520 ms latency and 234 B body size. The 'Body' tab is also visible, showing a 'Pretty' view of the response.

| Key | Value | Bulk Edit |
|-----|-------|-----------|
| Key | Value |           |

200 OK 520 ms 234 B

```
1 {  
2   "id": 14,  
3   "mark": "bmw",  
4   "model": "e38",  
5   "yearOfRelease": 2023  
6 }
```

### 3. *docker compose down -v*:

```
PS C:\Users\maxpo\devops-tms\homework22-23-24> docker compose down -v
[+] Running 6/6
✔ Container nginx Removed
✔ Container cars-api Removed
✔ Container mysqlDb Removed
✔ Volume homework22-23-24_datafiles Removed
✔ Network homework22-23-24_frontend Removed
✔ Network homework22-23-24_backend Removed
```

`docker compose down -v` останавливает и удаляет контейнеры, сети и все именованные тома, полностью уничтожая персистентные данные. Нельзя выполнять эту команду в production или на любых важных окружениях без свежего бэкапа, а также если вы просто хотите перезапустить сервисы — потому что данные БД исчезнут безвозвратно. Можно использовать только в локальной разработке для полного сброса окружения до чистого состояния (например, перед прогоном тестов с нуля).